You can see this works by first calling the rake task, `rails north_by:sell_out_shows` and then opening up a second browser window and hitting `http://localhost:3000/sold_out_concerts`. The new window will display a list of sold-out concerts. Simultaneously, the schedule page you already have open will have the newly updated concert displays change to “Sold Out”. Please note that this example is slightly faked; ideally we’d update the concert display every time a ticket was sold, which we’ll get to in a moment. In a real application, we’d set the sold-out state as we know it in the Rails view, and the ActionCable workflow would show updates.

Here’s the full workflow:

- The web request is routed to the `show` method of the `SoldOutConcertSource`. (We happen to have a `broadcast` method call there; it could be anywhere.)
- The `broadcast` call causes ActionCable to send data to any web browser that has subscribed to that stream. (Our client in the original browser happens to be such a subscriber and receives that data.)
- The callback method that we defined under the name `received` when we subscribed is invoked with the new data.
- That callback method does whatever it does. In our case, it signals to interested client code that the list of sold-out concerts has changed, and one bit of interested client code changes the browser DOM in response.

That’s actually kind of a lot of interaction that we got with only a handful of lines of code.

Now let’s build some ActionCable into our React page and make it interactive in both directions.